

04 — Operator e Operator Framework

Modulo: 1 — Introduzione e Architettura

Versione: OpenShift 4.x su Azure ARO

Obiettivi di Apprendimento

Al termine di questa lezione, il partecipante sarà in grado di:

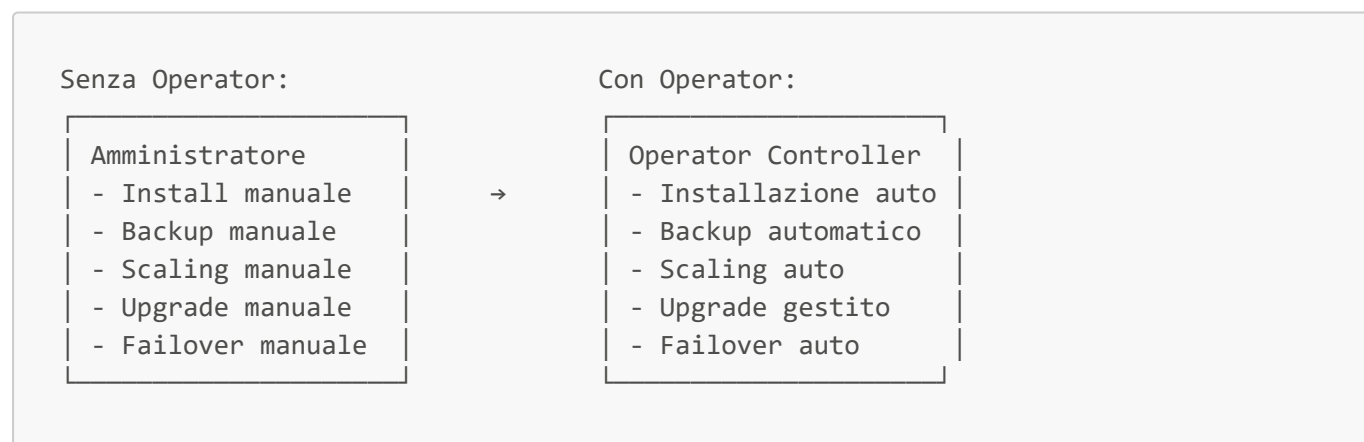
- Comprendere il pattern Operator e il problema che risolve
 - Distinguere CRD, CR e Operator Controller
 - Utilizzare l'Operator Lifecycle Manager (OLM) per installare e gestire Operator
 - Identificare gli Operator predefiniti di OCP e il loro stato
 - Eseguire operazioni pratiche con Operator su un cluster ARO
-

1. Cos'è un Operator?

Un **Operator** è un'applicazione software che estende l'API di Kubernetes per gestire applicazioni complesse (stateful) in modo automatizzato, replicando la conoscenza operativa di un amministratore esperto.

Il Problema che Risolve

Kubernetes gestisce bene le applicazioni **stateless** (Deployment, ReplicaSet). Per applicazioni **stateful** complesse (database, middleware, broker) — che richiedono procedure specifiche di installazione, backup, scaling, failover, upgrade — Kubernetes da solo non è sufficiente.



Il Pattern Operator

Un Operator è composto da tre elementi:

1. **CRD (Custom Resource Definition)** — estende l'API Kubernetes con nuovi tipi di oggetti
 2. **CR (Custom Resource)** — istanza della CRD che rappresenta lo stato desiderato
 3. **Controller** — loop di riconciliazione che porta il cluster allo stato desiderato
-

2. CRD, CR e Controller

2.1 Custom Resource Definition (CRD)

La CRD definisce un **nuovo tipo di oggetto** Kubernetes, con il suo schema e le sue proprietà:

```
# Esempio: CRD per un cluster MongoDB (semplificato)
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: mongodbcommunity.mongodbcommunity.mongodb.com
spec:
  group: mongodbcommunity.mongodb.com
  names:
    kind: MongoDBCommunity
    plural: mongodbcommunity
    singular: mongodbcommunity
  scope: Namespaced
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              members:
                type: integer
            type:
              type: string
              enum: [ReplicaSet, Standalone]
          version:
            type: string
```

```
# Visualizzare le CRD installate nel cluster
oc get crd
# Output atteso (prime righe):
# NAME                                     CREATED AT
# alertmanagerconfigs.monitoring.coreos.com 2024-01-10T10:00:00Z
# alertmanagers.monitoring.coreos.com      2024-01-10T10:00:00Z
# apirequestcounts.apiserver.openshift.io   2024-01-10T10:00:00Z
# authentications.operator.openshift.io     2024-01-10T10:00:00Z
# ...

# Visualizzare i dettagli di una CRD
oc describe crd clusterversions.config.openshift.io
```

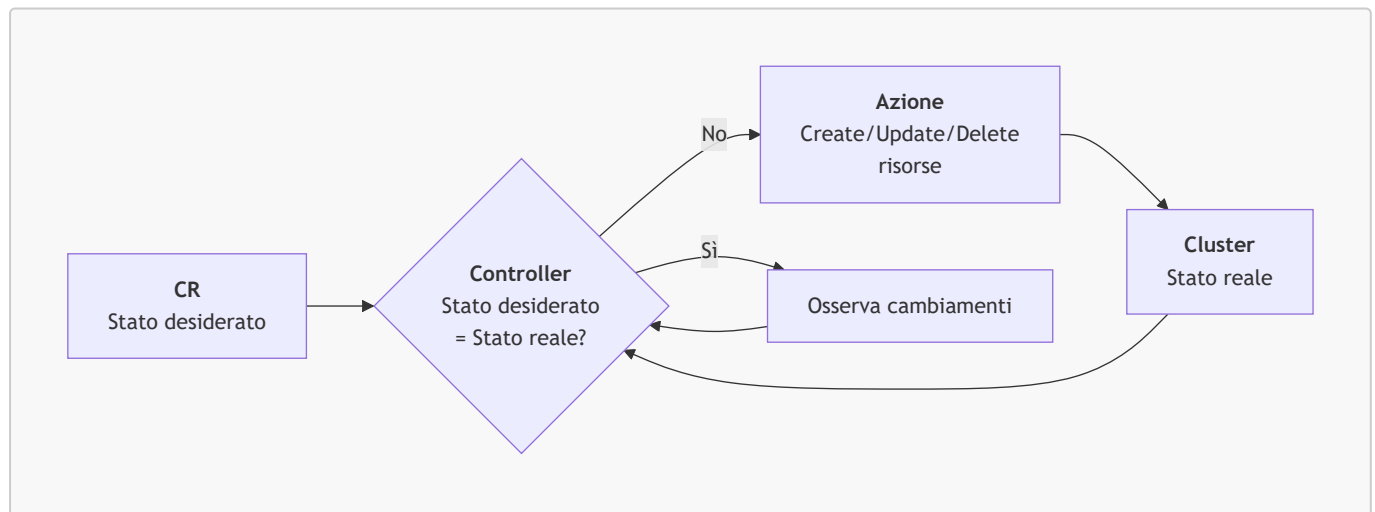
2.2 Custom Resource (CR)

La CR è un'**istanza** della CRD che l'utente crea per dichiarare lo stato desiderato:

```
# Esempio: CR per richiedere un cluster MongoDB
apiVersion: mongodbcommunity.mongodb.com/v1
kind: MongoDBCommunity
metadata:
  name: my-mongodb
  namespace: mongodb
spec:
  members: 3
  type: ReplicaSet
  version: "6.0.5"
  security:
    authentication:
      modes: ["SCRAM"]
  users:
    - name: admin
      db: admin
      passwordSecretRef:
        name: admin-password
      roles:
        - name: clusterAdmin
          db: admin
```

2.3 Controller (Reconciliation Loop)

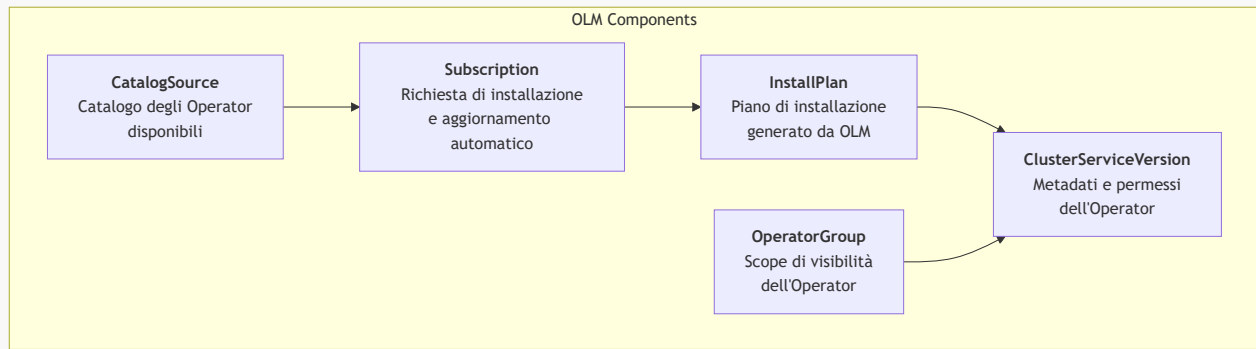
Il Controller osserva continuamente le CR e porta il cluster allo stato desiderato:



3. Operator Lifecycle Manager (OLM)

L'**OLM** è il componente di OpenShift che gestisce l'installazione, l'aggiornamento e la gestione del ciclo di vita degli Operator nel cluster.

Componenti di OLM



CatalogSource

Il **CatalogSource** è un catalogo di Operator disponibili per l'installazione:

```
# Visualizzare i CatalogSource disponibili
oc get catalogsource -n openshift-marketplace
# Output atteso:
# NAME                                DISPLAY                                TYPE    PUBLISHER    AGE
# certified-operators                 Certified Operators                    grpc    Red Hat      10d
# community-operators                 Community Operators                    grpc    OperatorHub  10d
# redhat-marketplace                  Red Hat Marketplace                    grpc    Red Hat      10d
# redhat-operators                    Red Hat Operators                      grpc    Red Hat      10d
```

Subscription

La **Subscription** avvia l'installazione di un Operator e gestisce gli aggiornamenti automatici:

```
# Esempio: installare l'Operator Prometheus tramite OLM
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: prometheus
  namespace: openshift-operators
spec:
  channel: beta
  name: prometheus
  source: community-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic # o Manual
```

```
# Applicare la Subscription
oc apply -f prometheus-subscription.yaml

# Verificare lo stato dell'installazione
oc get subscription -n openshift-operators
```

```
# Output atteso:
# NAME          PACKAGE          SOURCE              CHANNEL
# prometheus    prometheus       community-operators beta

# Verificare l'InstallPlan generato
oc get installplan -n openshift-operators
# Output atteso:
# NAME          CSV              APPROVAL    APPROVED
# install-xxxx  prometheusoperator.v0.70    Automatic   true
```

ClusterServiceVersion (CSV)

Il **CSV** contiene i metadati dell'Operator installato: versione, permessi RBAC, CRD gestite, immagine del controller:

```
# Verificare gli Operator installati (CSV)
oc get csv -n openshift-operators
# Output atteso:
# NAME                                DISPLAY          VERSION    REPLACES    PHASE
# prometheusoperator.v0.70.0          Prometheus       0.70.0                    Succeeded

# Dettagli di un CSV
oc describe csv prometheusoperator.v0.70.0 -n openshift-operators
```

4. Installare un Operator tramite Console Web

1. Navigare su **Operators** → **OperatorHub** nella console web
2. Cercare l'Operator desiderato (es. "Elasticsearch")
3. Cliccare su **Install**
4. Scegliere:
 - **Update channel** (versione/canale)
 - **Installation mode** (tutti i namespace o namespace specifico)
 - **Approval strategy** (Automatic / Manual)
5. Cliccare **Install** e attendere **Phase: Succeeded**

5. Operator Predefiniti di OCP

OpenShift 4.x include numerosi Operator predefiniti che gestiscono i componenti interni del cluster:

Cluster Operator (operatori core)

```
# Visualizzare tutti i Cluster Operator
oc get clusteroperator
# Output atteso:
# NAME                                VERSION    AVAILABLE    PROGRESSING
# DEGRADED
```

# authentication	4.14.x	True	False
False			
# baremetal	4.14.x	True	False
False			
# cloud-controller-manager	4.14.x	True	False
False			
# cloud-credential	4.14.x	True	False
False			
# cluster-autoscaler	4.14.x	True	False
False			
# config-operator	4.14.x	True	False
False			
# console	4.14.x	True	False
False			
# dns	4.14.x	True	False
False			
# etcd	4.14.x	True	False
False			
# image-registry	4.14.x	True	False
False			
# ingress	4.14.x	True	False
False			
# kube-apiserver	4.14.x	True	False
False			
# kube-controller-manager	4.14.x	True	False
False			
# kube-scheduler	4.14.x	True	False
False			
# machine-api	4.14.x	True	False
False			
# machine-approver	4.14.x	True	False
False			
# machine-config	4.14.x	True	False
False			
# monitoring	4.14.x	True	False
False			
# network	4.14.x	True	False
False			
# node-tuning	4.14.x	True	False
False			
# openshift-apiserver	4.14.x	True	False
False			
# operator-lifecycle-manager	4.14.x	True	False
False			
# operator-lifecycle-manager-catalog	4.14.x	True	False
False			
# storage	4.14.x	True	False
False			

Operator più Utilizzati da OperatorHub

Operator	Scopo	Canale consigliato
OpenShift Pipelines	CI/CD con Tekton	latest
OpenShift GitOps	GitOps con ArgoCD	latest
OpenShift Serverless	Knative serving/eventing	stable
OpenShift Service Mesh	Istio + Kiali + Jaeger	stable
Elasticsearch Operator	Log storage per OpenShift Logging	stable
Loki Operator	Log storage alternativo	stable
Cert-Manager	Gestione certificati TLS	stable
External Secrets Operator	Integrazione con Azure Key Vault	stable
KEDA	Event-driven autoscaling	stable

6. Esempio Completo: Installare OpenShift Pipelines

```
# 1. Creare la Subscription per OpenShift Pipelines
cat <<EOF | oc apply -f -
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: openshift-pipelines-operator
  namespace: openshift-operators
spec:
  channel: latest
  name: openshift-pipelines-operator-rh
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
EOF

# 2. Attendere il completamento dell'installazione
oc wait --for=condition=Succeeded \
  csv -l operators.coreos.com/openshift-pipelines-operator-rh.openshift-operators \
  -n openshift-operators \
  --timeout=300s

# 3. Verificare l'installazione
oc get csv -n openshift-operators | grep pipelines
# Output atteso:
# openshift-pipelines-operator-rh.v1.13.x    Red Hat OpenShift Pipelines    1.13.x
Succeeded

# 4. Verificare le CRD installate dall'Operator
oc get crd | grep tekton
# Output atteso:
# clustertasks.tekton.dev                    2024-01-10T10:00:00Z
```

```
# pipelines.tekton.dev                2024-01-10T10:00:00Z
# pipelineruns.tekton.dev            2024-01-10T10:00:00Z
# tasks.tekton.dev                  2024-01-10T10:00:00Z
# taskruns.tekton.dev               2024-01-10T10:00:00Z

# 5. Creare una Pipeline di esempio
cat <<EOF | oc apply -f - -n my-project
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: hello-pipeline
spec:
  tasks:
  - name: say-hello
    taskSpec:
      steps:
      - name: echo
        image: registry.access.redhat.com/ubi9/ubi-minimal
        command: [echo]
        args: ["Hello from OpenShift Pipelines!"]
EOF

# 6. Avviare la Pipeline
oc create pipelinerun hello-run \
  --pipeline=hello-pipeline \
  -n my-project

# Verificare l'output
oc logs -l tekton.dev/pipelineRun=hello-run --all-containers -n my-project
```

7. Note Specifiche per Azure ARO

```
# Su ARO, l'External Secrets Operator permette di sincronizzare
# i segreti da Azure Key Vault direttamente come Secret Kubernetes

# Installare External Secrets Operator
cat <<EOF | oc apply -f -
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: external-secrets-operator
  namespace: openshift-operators
spec:
  channel: stable
  name: external-secrets-operator
  source: community-operators
  sourceNamespace: openshift-marketplace
EOF

# Configurare la SecretStore per Azure Key Vault
```



```
cat <<EOF | oc apply -f -
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: azure-keyvault-store
  namespace: my-project
spec:
  provider:
    azurekv:
      url: "https://my-keyvault.vault.azure.net"
      authType: WorkloadIdentity
EOF
```

Nota ARO: Alcune restrizioni OLM sono applicate da Red Hat/Microsoft in ARO. Gli Operator che richiedono accesso privilegiato ai Master Node potrebbero non essere installabili. Verificare sempre la compatibilità ARO sul [Red Hat Ecosystem Catalog](#).

8. Troubleshooting degli Operator

```
# Verificare lo stato di tutti gli Operator (cercare DEGRADED=True)
oc get clusteroperator | grep -v "True.*False.*False"

# Dettagli su un Operator in stato degraded
oc describe clusteroperator monitoring

# Verificare i log del Pod dell'Operator
oc get pods -n openshift-operators
oc logs deployment/my-operator-controller-manager -n openshift-operators

# Forzare la riconciliazione di un Operator (ricreare il Pod)
oc delete pod -l name=my-operator -n openshift-operators

# Visualizzare gli eventi recenti in un namespace
oc get events -n openshift-operators --sort-by='.lastTimestamp' | tail -20
```

9. Riepilogo dei Punti Chiave

- Un **Operator** automatizza la gestione operativa di applicazioni complesse, codificando la conoscenza di un amministratore esperto
- I tre elementi fondamentali: **CRD** (tipo di oggetto), **CR** (istanza/stato desiderato), **Controller** (loop di riconciliazione)
- L'**OLM** (Operator Lifecycle Manager) gestisce installazione, versioning e aggiornamento degli Operator in OCP
- La **Subscription** specifica quale Operator installare e il canale di aggiornamento
- OCP include decine di **Cluster Operator** predefiniti che gestiscono ogni componente interno del cluster
- In **ARO** alcuni Operator privilegiati potrebbero non essere supportati: verificare sempre la compatibilità

10. Domande di Verifica

1. Qual è la differenza tra una **CRD** e una **CR**? Fai un esempio concreto.
 2. Cosa fa il Controller di un Operator quando trova una discrepanza tra stato desiderato e stato reale?
 3. Qual è il vantaggio di usare `installPlanApproval: Manual` invece di `Automatic` in una Subscription?
 4. Come verifichi che tutti i Cluster Operator di un cluster OCP siano in stato sano?
 5. In Azure ARO vuoi leggere i segreti da Azure Key Vault come Secret Kubernetes. Quale Operator useresti e perché?
-

Riferimenti

- [Operator Framework - Red Hat](#)
- [Operator Lifecycle Manager Documentation](#)
- [OperatorHub.io](#)
- [Red Hat Ecosystem Catalog \(compatibilità ARO\)](#)
- [External Secrets Operator + Azure Key Vault](#)
- [OpenShift Pipelines \(Tekton\)](#)